

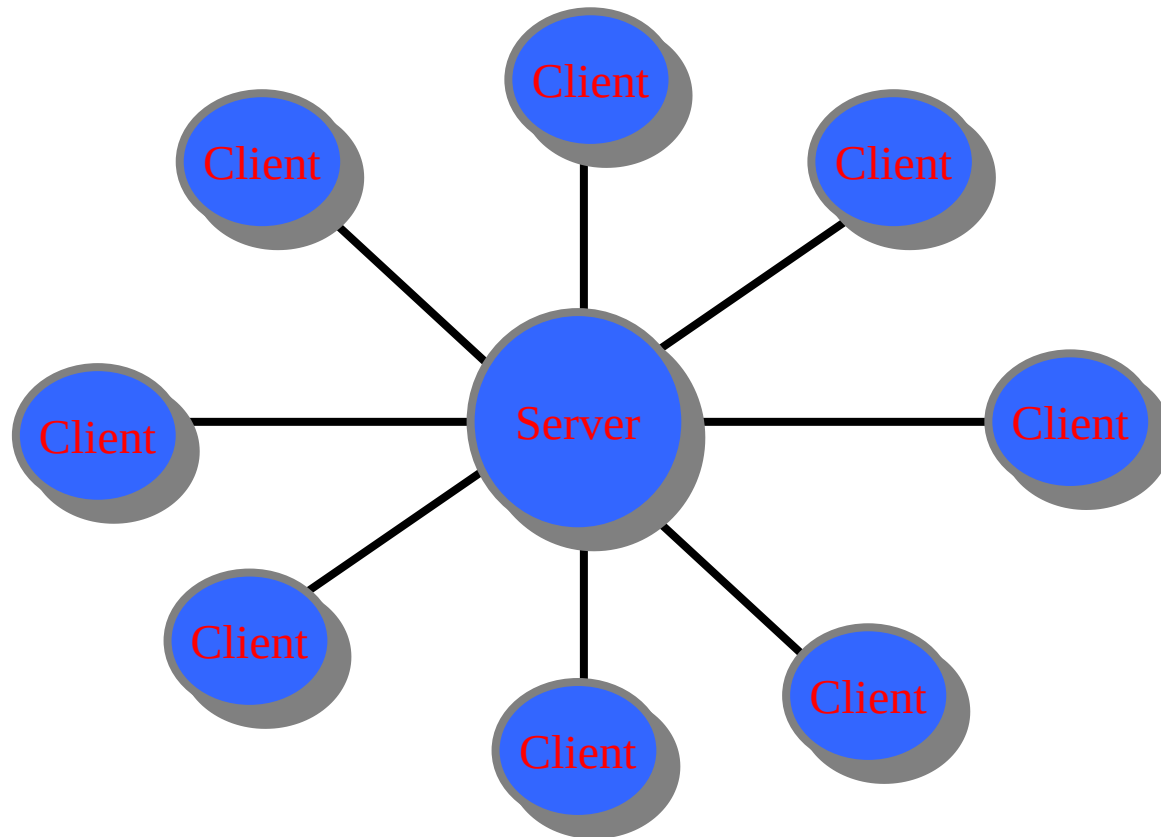
Bài 06:

TCP Socket

Nội dung

- ❑ Mô hình Client-Server
- ❑ Mô hình truyền tin Socket
- ❑ TCP Socket client - Socket
- ❑ TCP Socket server - ServerSocket

Mô hình Client/Server



Mô hình Client/Server

- ❑ Các ứng dụng mạng thường hoạt động theo mô hình client/server.
Ví dụ:
 - Web server / web client
 - Mail server / mail client
 - FTP server / ftp client
 - ...
- ❑ Mô hình này gồm 1 chương trình đóng vai trò client và 1 chương trình đóng vai trò server.
 - Chương trình server: cung cấp dịch vụ, lắng nghe yêu cầu từ client, xử lý và trả kết quả tương ứng.
 - Chương trình client: gửi yêu cầu dịch vụ đến server và đợi đáp trả từ server.
- ❑ Quá trình trao đổi dữ liệu giữa client/server gồm: Truyền 1 yêu cầu từ client tới server, yêu cầu được server xử lý, truyền đáp ứng cho client.

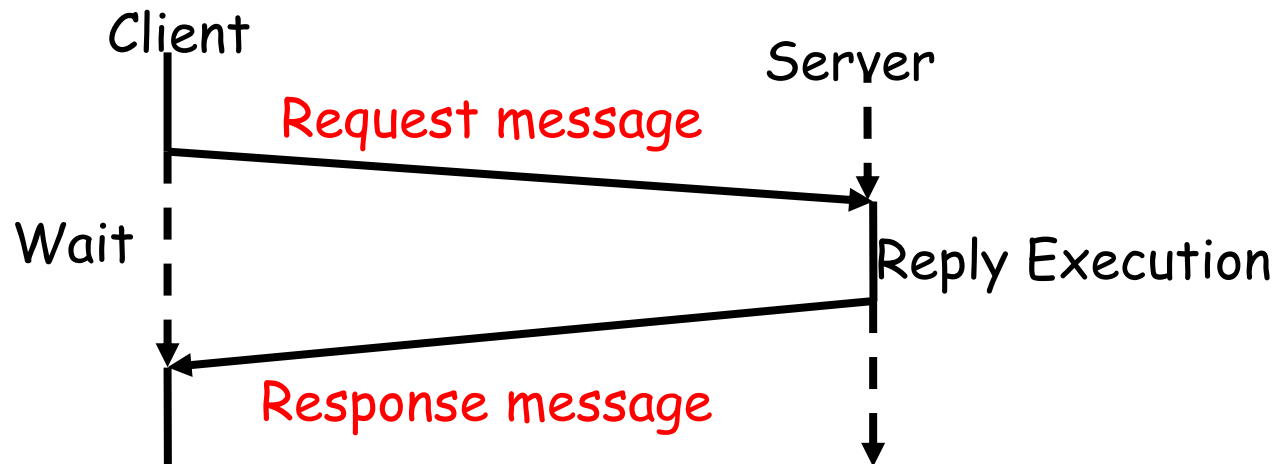
Mô hình Client/Server

Server

- Chạy trước
- Luôn đợi thiết lập kết nối từ client
- Đáp trả yêu cầu cho client tương ứng.

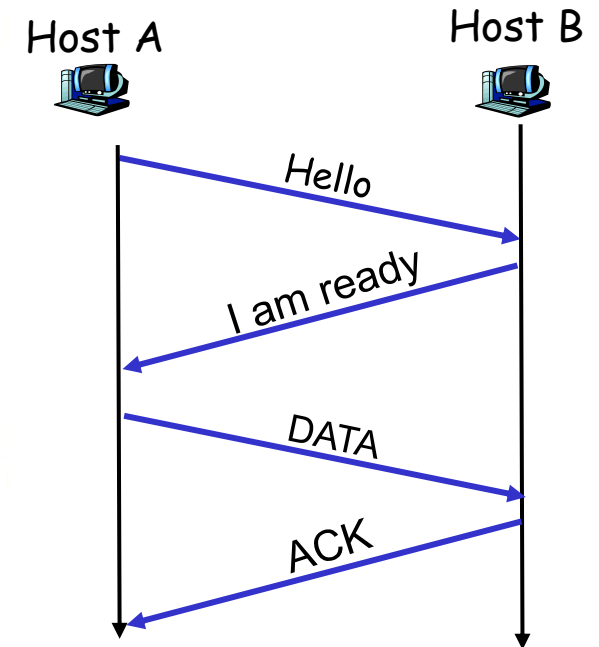
Client

- Chạy sau
- Thiết lập kết nối tới Server và gửi yêu cầu.
- Đợi đáp trả từ Server.

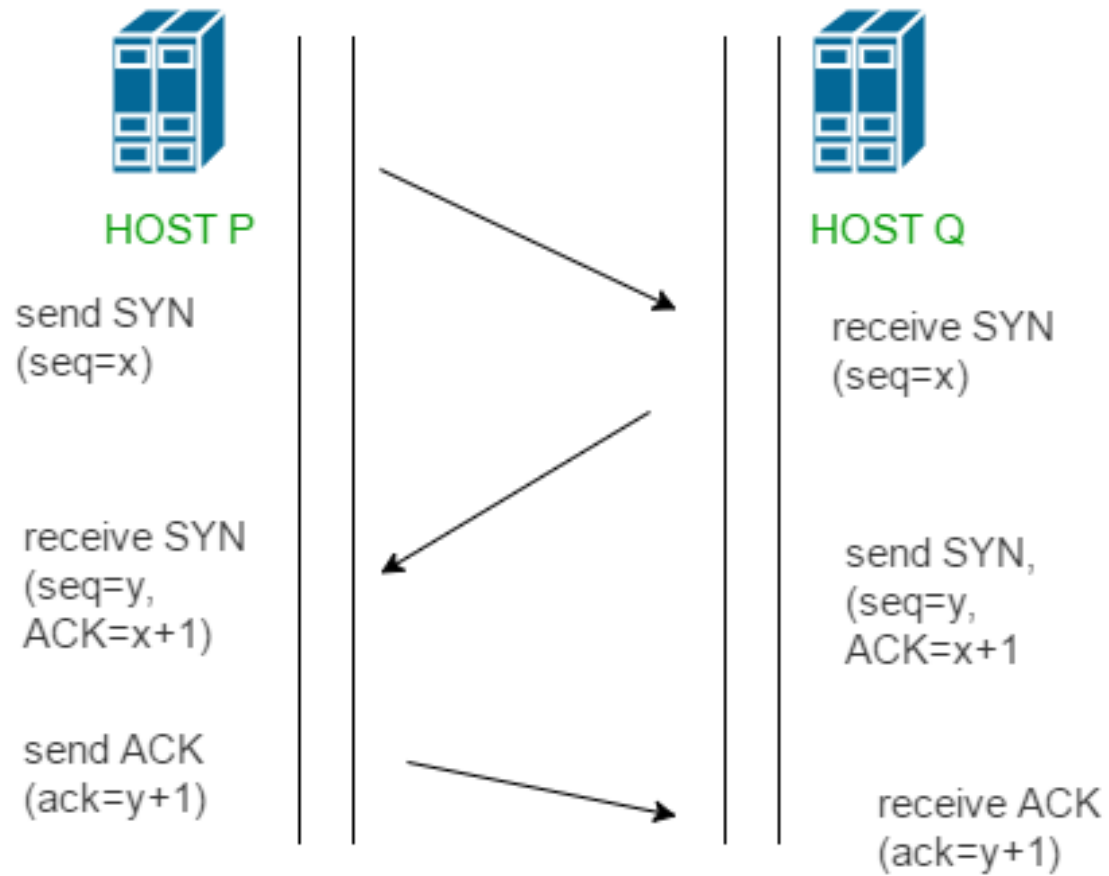


TCP

+	Bit 0 - 3	4 - 9	10 - 15	16 - 31
0	Source Port			Destination Port
32	Sequence Number			
64	Acknowledgement Number			
96	Data Offset	Reserved	Flags	Window
128	Checksum			Urgent Pointer
160	Options (optional)			
160/192+	Data			



TCP



TCP

Trường	Mô tả
source port	Số hiệu cổng của nguồn
destination port	Số hiệu cổng đích
Sequence Number	Số thứ tự được tạo ra bởi nguồn
Acknowledge Number	Cho biết dữ liệu được nhận thành công.
Data offset	Các chi tiết về nơi dữ liệu gói tin bắt đầu
Reserved	Dự phòng
Flags	chỉ ra rằng gói tin cuối cùng hoặc gói khẩn cấp
Window	chỉ ra kích thước của vùng đệm nhận.
Checksum	xác định xem gói tin có bị hỏng không
Urgent Pointer	thông báo cho phía nhận biết có dữ liệu khẩn
Options	vùng dự phòng cho việc thiết lập trong tương lai
Padding	chỉ ra rằng dữ liệu kết thúc trong vòng 32 bit.

Số hiệu cổng

❑ Có 3 loại (netstat -a)

- số hiệu cổng hệ thống (0-1023). Các giao thức nổi tiếng có các số hiệu cổng nằm trong khoảng này.
- Các số hiệu cổng người dùng (1024-49151). Các ứng dụng server của bạn sẽ nhận một trong các số này làm cổng, hoặc bạn có thể đăng ký số hiệu cổng với IANA .
- Các số hiệu cổng riêng và động (49152-65535).

Mô hình truyền tin socket

- ❑ Chương trình client và server sử dụng giao thức vận chuyển (TCP/UDP) để gửi và nhận dữ liệu.
- ❑ Dữ liệu được truyền trên mạng dưới dạng các gói tin (packet) có kích thước hữu hạn gọi là datagram. Mỗi datagram chứa header và payload.
 - Header: địa chỉ cổng nguồn, đích, các thông tin khác để đảm bảo độ tin cậy truyền tin.
 - Payload: chứa dữ liệu.
- ❑ Trong quá trình truyền tin có thể bị mất, bị hỏng hoặc các gói tin đến không theo đúng trình tự.
- ❑ Khái niệm lập trình socket cho phép người lập trình xem một liên kết mạng như là một luồng mà có thể đọc hay ghi dữ liệu từ luồng này.
- ❑ Các socket che dấu người lập trình khỏi các chi tiết mức thấp của mạng như: kiểu đường truyền, kích thước gói, yêu cầu truyền lại gói, các địa chỉ mạng, ...

Mô hình truyền tin socket

Một socket có thể thực hiện bảy thao tác cơ bản:

1. Kết nối với một máy ở xa
2. Gửi dữ liệu
3. Nhận dữ liệu
4. Ngắt kết nối
5. Gán cổng
6. Nghe dữ liệu đến
7. Chấp nhận liên kết từ các máy ở xa trên cổng đã được gán

Mô hình truyền tin socket

Có nhiều kiểu socket khác nhau tương ứng với kiểu giao thức được sử dụng. Đối với chồng giao thức TCP/IP, có hai kiểu socket chính được sử dụng:

- Stream socket: sử dụng giao thức TCP
 - Socket
 - ServerSocket (dành riêng cho server)
- Datagram socket: sử dụng giao thức UDP
 - DatagramPacket
 - DatagramSocket

Socket

- ❑ `java.net.Socket`
- ❑ Là đối tượng biểu diễn logic điểm đầu cuối của luồng giao tiếp giữa hai host.
- ❑ Client thiết lập giao tiếp với server và đợi sự đáp trả từ server. Các socket cho client thường được sử dụng theo các bước sau:
 - Tạo socket với địa chỉ IP và số hiệu cổng của máy server và thực hiện liên kết kết với socket server. Các liên kết này là song công (full-duplex).
 - Sau khi liên kết được thiết lập, thực hiện việc nhập, xuất trên luồng dữ liệu.

Socket

Phương thức khởi tạo

<pre>public Socket (String host, int port) throws UnknownHostException, IOException</pre>	Hàm này tạo một socket TCP với host và cổng xác định, và thực hiện liên kết với host ở xa.
<pre>public Socket (InetAddress host, int port) throws IOException</pre>	Tạo một socket TCP với thông tin là địa chỉ của một host được xác định bởi một đối tượng InetAddress và số hiệu cổng port, sau đó nó thực hiện kết nối tới host.
<pre>public Socket (String host, int port, InetAddress interface, int localPort) throws IOException, UnknownHostException</pre>	Tạo ra một socket với thông tin là địa chỉ IP được biểu diễn bởi một đối tượng String và một số hiệu cổng và thực hiện kết nối tới host đó. Socket kết nối tới host ở xa thông qua một giao tiếp mạng và số hiệu cổng cục bộ. Nếu localPort bằng 0 thì Java sẽ lựa chọn một cổng ngẫu nhiên có sẵn nằm trong khoảng từ 1024 đến 65535.
<pre>public Socket (InetAddress host, int port, InetAddress interface, int localPort) throws IOException, UnknownHostException</pre>	Constructor chỉ khác constructor trên ở chỗ địa chỉ của host lúc này được biểu diễn bởi một đối tượng InetAddress.

Socket

Phương thức giao tiếp

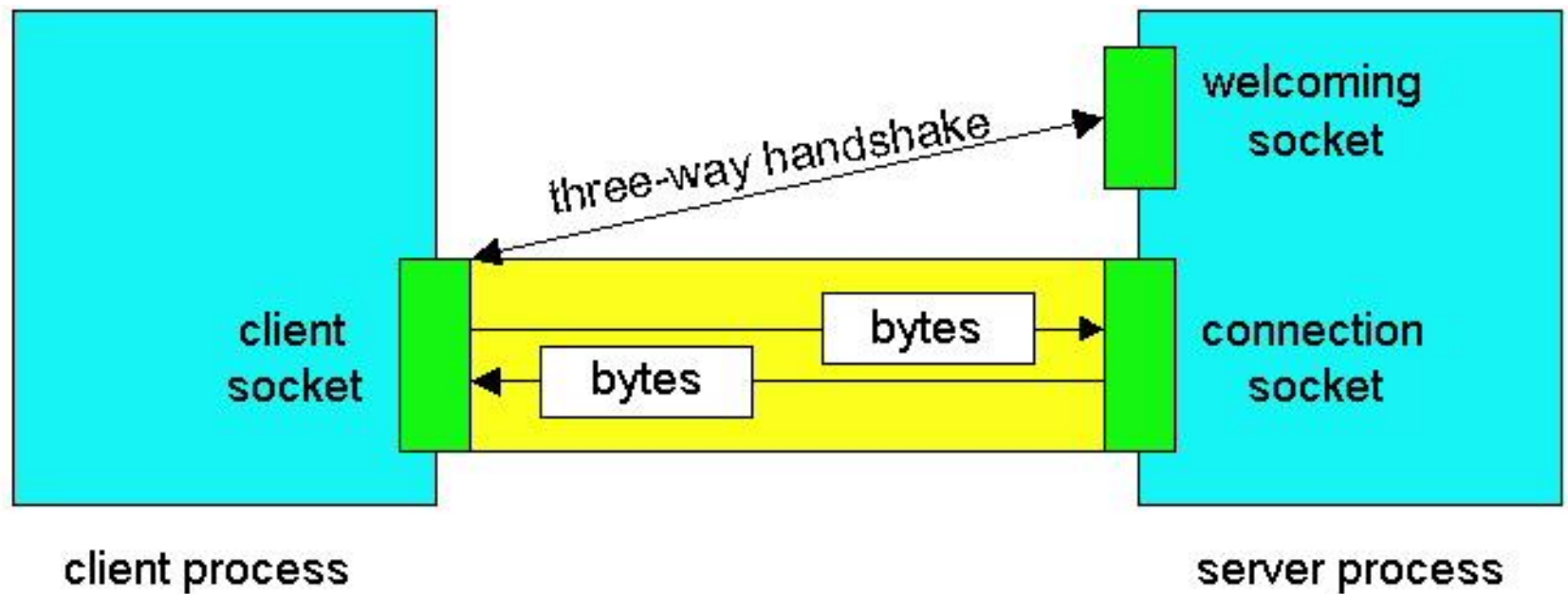
<code>public InetAddress getInetAddress ()</code>	Trả lại địa chỉ máy đích bằng đối tượng <code>InetAddress</code> .
<code>public int getPort ()</code>	Lấy số hiệu cổng dịch vụ của máy đích.
<code>public int getLocalPort ()</code>	Trả lại địa chỉ cổng sử dụng ở máy cục bộ.
<code>public InetAddress getLocalAddress ()</code>	Trả lại địa chỉ mạng của máy cục bộ dùng để giao tiếp với máy ở xa.
<code>public InputStream getInputStream () throws IOException</code>	Trả về một luồng nhập để đọc dữ liệu từ một socket vào chương trình.
<code>Public OutputStream getOutputStream () throws IOException</code>	Trả về một luồng xuất thô để ghi dữ liệu từ ứng dụng ra đầu cuối của một socket.

Socket

Phương thức đóng socket

<code>public void close() throws IOException</code>	Đóng luồng kết nối lại.
<code>public void shutdownInput() throws IOException</code>	Đóng một luồng nhập.
<code>public void shutdownOutput() throws IOException</code>	Đóng luồng xuất.
<code>public boolean isOutputShutdown()</code>	Kiểm tra luồng xuất đóng chưa.
<code>public boolean isInputShutdown()</code>	Kiểm tra luồng nhập đóng chưa.

Socket



Socket client

- ❑ Tạo đối tượng socket kết nối tới server

```
Socket s = new Socket ("java.sun.com", 8189)
```

- ❑ Lấy luồng gửi và đọc dữ liệu từ đối tượng socket.

```
InputStream ins = s.getInputStream ();
```

```
OutputStream outs = s.getOutputStream();
```

- ❑ Gắn đối tượng đọc viết có định dạng vào luồng

```
BufferedReader in = new BufferedReader(new InputStreamReader(ins));
```

```
PrintWriter out = new PrintWriter(outs, true);
```

- ❑ Tiến hành đọc viết dữ liệu

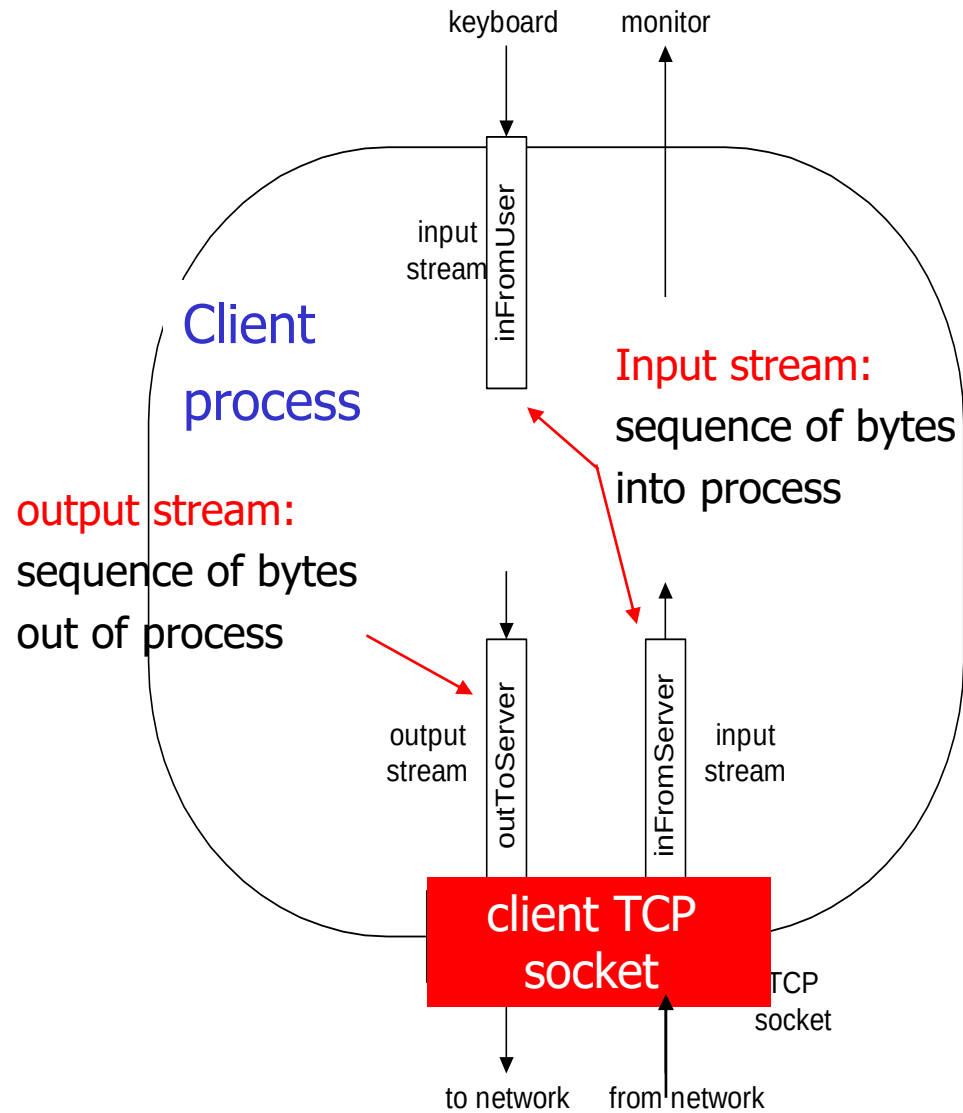
```
String str = in.readLine();
```

```
out.println ( "Echo:" + str + "\r");
```

- ❑ Đóng kết nối:

```
s.close();
```

Socket client



Ví dụ Socket client

RFC867 - Daytime Protocol <http://www.faqs.org/rfcs/rfc867.html>

```
17 public class TimeClientEx1 {
18
19     public static void main(String[] args) throws IOException{
20         // TODO code application logic here
21         String hostname = "time.nist.gov";
22
23         try {
24             Socket theSocket = new Socket(hostname, 13);
25             InputStream timeStream = theSocket.getInputStream();
26              StringBuffer time = new StringBuffer();
27             int c;
28             while ((c = timeStream.read()) != -1) {
29                 time.append((char) c);
30             }
31             String timeString = time.toString().trim();
32             System.out.println("It is " + timeString + " at " + hostname);
33         } catch (UnknownHostException ex) {
34             System.err.println(ex);
35         } catch (IOException ex) {
36             System.err.println(ex);
37         }
38     }
39 }
```

ServerSocket

- ❑ `java.net.ServerSocket`
- ❑ Lớp `ServerSocket` có mọi thứ để viết server bằng Java:
 - Các phương thức khởi tạo
 - Lắng nghe và chấp nhận kết nối trên cổng đã mở
 - Đóng kết nối

ServerSocket

Phương thức khởi tạo

<pre>public ServerSocket(int port) throws IOException, BindException</pre>	Tạo một socket cho server trên cổng xác định.
<pre>public ServerSocket(int port, int queuelength, InetAddress bindAddress) throws IOException</pre>	Tạo một đối tượng ServerSocket trên cổng xác định với chiều dài hàng đợi xác định. ServerSocket chỉ gán cho địa chỉ IP cục bộ xác định. Constructor này hữu ích cho các server chạy trên các hệ thống có nhiều địa chỉ IP.

Constructor đưa ra ngoại lệ IOException nếu ta không thể tạo và gán socket cho cổng được yêu cầu. Ngoại lệ IOException phát sinh khi:

- Cổng đã được sử dụng
- Không có quyền hoặc cố liên kết với một cổng nằm giữa 0 và 1023.

ServerSocket

Chấp nhận và ngắt kết nối

<code>public Socket accept() throws IOException</code>	Phương thức này phong tỏa; nó dừng quá trình xử lý và đợi cho tới khi client được kết nối. Khi client thực sự kết nối, phương thức <code>accept()</code> trả về đối tượng <code>Socket</code> .
<code>public int getLocalHost()</code>	Các constructor <code>ServerSocket</code> cho phép ta nghe dữ liệu trên cổng không định trước bằng cách gán số 0 cho cổng. Phương thức này cho phép ta tìm ra cổng mà server đang nghe.
<code>public InetAddress getInetAddress()</code>	Phương thức này trả về địa chỉ được sử dụng bởi server (localhost).
<code>public void close() throws IOException</code>	Đóng socket.

ServerSocket

- ❑ Tạo đối tượng Socket Server với một cổng lắng nghe xác định

```
ServerSocket s = new ServerSocket (8189)
```

- ❑ Đợi lắng nghe yêu cầu/kết nối từ client và tạo ra Socket giao tiếp.

```
while(true) {  
    Socket new_s = s.accept();  
    doSomething(new_s);  
}
```

- ❑ Xử lý giao tiếp với Socket client dựa vào đối tượng new_s này sử dụng luồng nhập/xuất.

- ❑ Đóng kết nối giao tiếp với client :

```
s.close();
```

Ví dụ Time Server

```
5  L public class TimeServerEx2 {
6
7      public static void main(String[] args) {
8
9          int port = 2021;
10
11         try ( ServerSocket serverSocket = new ServerSocket(port)) {
12
13             System.out.println("Server is listening on port " + port);
14
15             while (true) {
16                 Socket socket = serverSocket.accept();
17
18                 System.out.println("New client connected");
19
20                 OutputStream output = socket.getOutputStream();
21                 PrintWriter writer = new PrintWriter(output, true);
22
23                 writer.println(new Date().toString());
24             }
25
26         } catch (IOException ex) {
27             System.out.println("Server exception: " + ex.getMessage());
28             ex.printStackTrace();
29         }
30     }
31 }
```

Ví dụ Time Client

```
16 public class TimeClientEx2 {
17
18     public static void main(String[] args) {
19         String hostname = "127.0.0.1";
20         int port = 2021;
21
22         try ( Socket socket = new Socket(hostname, port)) {
23
24             InputStream input = socket.getInputStream();
25             BufferedReader reader = new BufferedReader(new InputStreamReader(input));
26
27             String time = reader.readLine();
28
29             System.out.println(time);
30
31         } catch (UnknownHostException ex) {
32
33             System.out.println("Server not found: " + ex.getMessage());
34
35         } catch (IOException ex) {
36
37             System.out.println("I/O error: " + ex.getMessage());
38         }
39     }
40 }
```

Bài tập 1: BAI06-BT01

Chương trình client liên kết với một server sử dụng TCP socket. Người dùng nhập vào một dòng ký tự từ bàn phím và gửi dữ liệu đến server.

- Server nhận được dữ liệu thì chuyển thành các ký tự hoa và gửi trả lại client.
- Client nhận chuỗi dữ liệu & hiển thị ra màn hình.

TCPClient.java

```
11 public class TCPClientEx1 {
12
13     public static void main(String[] args) {
14         String hostname = "localhost";
15         if (args.length > 0) {
16             hostname = args[0];
17         }
18         PrintWriter pw = null;
19         BufferedReader br = null;
20         try {
21             Socket s = new Socket(hostname, 2007);
22             br = new BufferedReader(new InputStreamReader(s.getInputStream()));
23             BufferedReader user;
24             user = new BufferedReader(new InputStreamReader(System.in));
25             pw = new PrintWriter(s.getOutputStream());
26             System.out.println("Da ket noi duoc voi server...");
27             while (true) {
28                 String st = user.readLine();
29                 if (st.equals("exit")) {
30                     break;
31                 }
32                 pw.println(st);
33                 pw.flush();
34                 System.out.println(br.readLine());
35             }
36         } catch (IOException e) {
37             System.err.println(e);
38         } finally {
39             try {
40                 if (br != null) {
41                     br.close();
42                 }
43                 if (pw != null) {
44                     pw.close();
45                 }
46             } catch (IOException e) {
47                 System.err.println(e);
48             }
49         }
50     }
51 }
```

TCP Server

```
11 public class TCPServerEx1 {
12
13     public final static int DEFAULT_PORT = 2007;
14
15     public static void main(String[] args) {
16         int port = DEFAULT_PORT;
17         try {
18             ServerSocket ss = new ServerSocket(port);
19             Socket s = null;
20             while (true) {
21                 try {
22                     s = ss.accept();
23                     PrintWriter pw;
24                     BufferedReader br;
25                     pw = new PrintWriter(new OutputStreamWriter(s.getOutputStream()));
26                     br = new BufferedReader(new InputStreamReader(s.getInputStream()));
27                     while (true) {
28                         String line = br.readLine();
29                         if (line.equals("exit")) {
30                             break;
31                         }
32                         String upper = line.toUpperCase();
33                         pw.println(upper);
34                         pw.flush();
35                     }
36                 } catch (IOException e) {
37                 } finally {
38                     try {
39                         if (s != null) {
40                             s.close();
41                         }
42                     } catch (IOException e) {
43                     }
44                 }
45             }
46         } catch (IOException e) {
47         }
48     }
49 }
```

Bài tập 2: BAI06-BT02

Viết chương trình giao tiếp giữa client và server sử dụng giao thức TCP, thực hiện các chức năng sau:

- Client truyền hai số nguyên và phép toán (cộng, trừ, nhân, chia) lên server.
- Server sau khi nhận được thì thực hiện phép toán giữa hai số nguyên và trả kết quả về cho client, client nhận lại kết quả và xuất ra màn hình.

TCPClient.java

```
19 public class TCPClientEx2 {
20
21     private static Socket socket = null;
22     private static PrintWriter out = null;
23
24     public static void main(String args[]) {
25         Socket socket = null;
26         PrintWriter out = null;
27         Scanner in = null;
28         int so1, so2;
29         String pheptoan, chuoi, ketqua;
30         in = new Scanner(System.in);
31         out = new PrintWriter(System.out, true);
32         out.println("Nhập số thứ nhất:");
33         so1 = in.nextInt();
34         out.println("Nhập số thứ hai:");
35         so2 = in.nextInt();
36         out.println("Nhập phép toán:");
37         pheptoan = in.next();
38         chuoi = so1 + "@" + pheptoan + "@" + so2;
39         try {
40             socket = new Socket("127.0.0.1", 1234);
41             out = new PrintWriter(socket.getOutputStream(), true);
42             in = new Scanner(socket.getInputStream());
43             out.println(chuoi);
44             ketqua = in.nextLine().trim();
45             System.out.println("Kết quả:" + ketqua);
46             socket.close();
47         } catch (IOException e) {
48             if (socket != null) {
49                 try {
50                     socket.close();
51                 } catch (IOException ex) {
52                 }
53             }
54         }
55     }
56 }
```

TCP Server

```
19 public class TCPServerEx2 {
20     public static void main(String[] args) throws IOException {
21         int PORT = 1234;
22         ServerSocket server = new ServerSocket(PORT);
23
24         System.out.println("Server listening");
25         try {
26             Socket socket;
27             while ((socket = server.accept()) != null) {
28                 Scanner in = new Scanner(socket.getInputStream());
29                 PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
30
31                 String chuoi = in.nextLine().trim();
32                 in = new Scanner(chuoi);
33                 in.useDelimiter("@");
34                 int so1 = in.nextInt();
35                 String pheptoan = in.next();
36                 int so2 = in.nextInt();
37                 switch (pheptoan) {
38                     case "+":
39                         out.println(so1 + so2);
40                         break;
41                     case "-":
42                         out.println(so1 - so2);
43                         break;
44                     case "*":
45                         out.println(so1 * so2);
46                         break;
47                     case "/":
48                         out.println((float)so1 / so2);
49                         break;
50                     default:
51                         break;
52                 }
53             }
54         } catch (Exception e) {
55             e.printStackTrace();
56         }
57     }
58 }
```

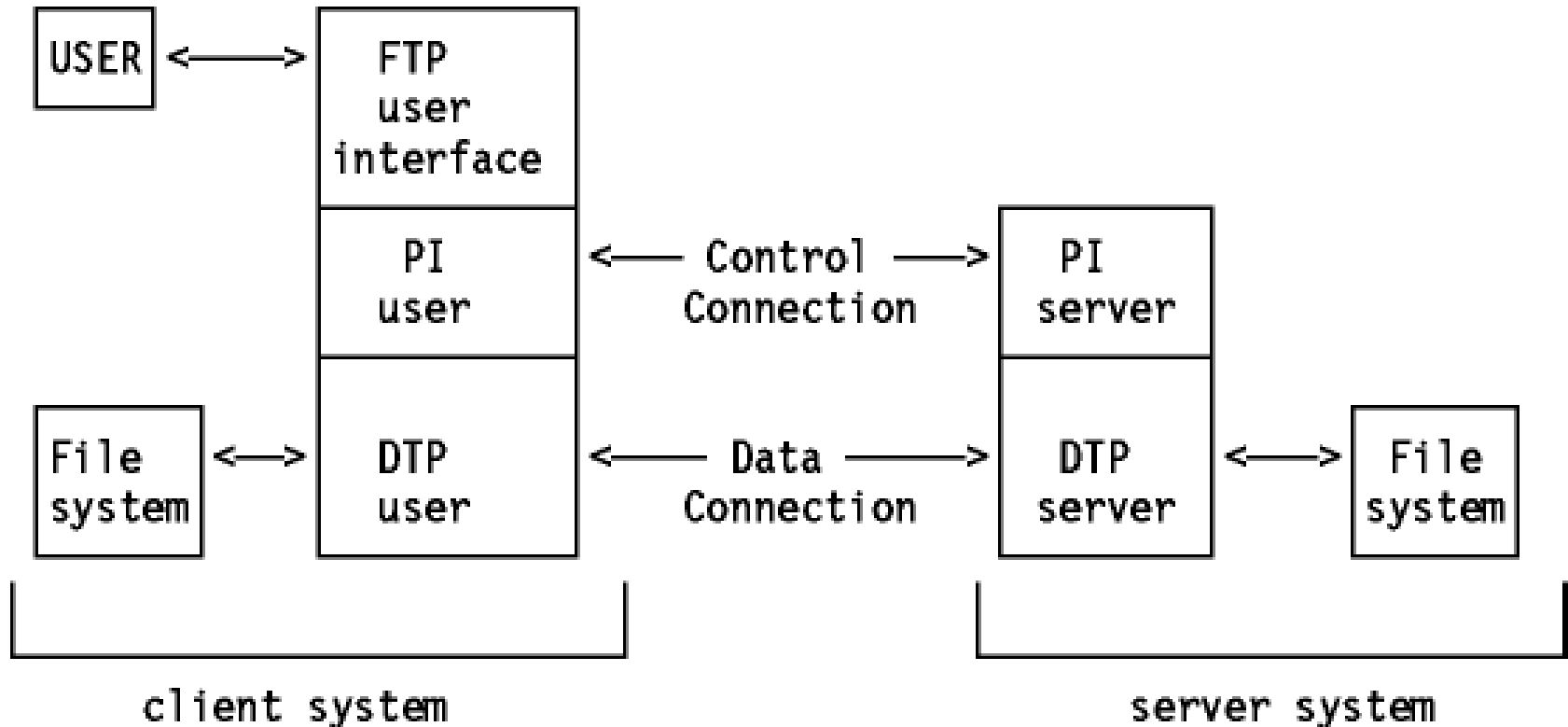
Bài tập 3: BAI06-BT03

Viết chương trình minh họa giao thức FTP cho phép client tạo kết nối gửi file lên server sử dụng TCP socket:

- Client: Gửi 1 file lên server.
- Server: Đọc dữ liệu từ client và ghi vào file.

Bài tập 4 (nâng cao):

BAI06-BT04



PI : protocol interpreter
DTP: data transfer process